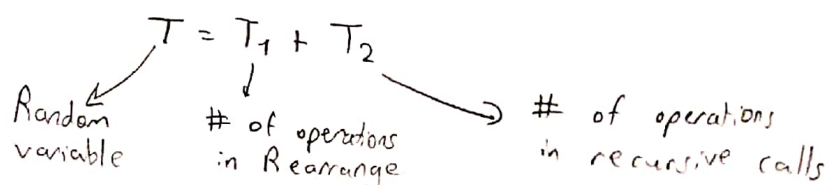


Quicksort

Average Case Analysis:

Assume that it is equally likely that the pivot element ($L[\text{low}]$) will be placed in any position after Rearrange.

[The proper position of the pivot is the important thing in this algorithm]



$$A(n) = E[T] = E[T_1] + E[T_2]$$

Annotations for $E[T_1]$:

- fixed $(n+1)$
- due to high-low + 2 comparisons

Annotation for $E[T_2]$:

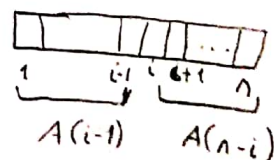
- depends on where the pivot has been placed

If the proper position of the pivot was $L[1]$, then the 2 sublists are $L[1:0]$ and $L[2:n]$

If the proper position of the pivot was $L[2]$, then the 2 sublists are $L[1:1]$ and $L[3:n]$

⊛ If the proper position of the pivot was $L[i]$, then the 2 sublists are $L[1:i-1]$ and $L[i+1:n]$

$$E[T_2] = \sum_{\text{position of the pivot } x} E[T_2 | \bar{x} = x] \cdot \underbrace{P(\bar{x} = x)}_{1/n}$$



$$A(n) = (n+1) + \sum_{i=1}^n E[T_2 | \bar{x} = i] \cdot \frac{1}{n}$$

$$A(n) = (n+1) + \sum_{i=1}^n [A(i-1) + A(n-i)] \frac{1}{n}$$

Full history recurrence relation

$$\begin{aligned} i=1 &\Rightarrow A(0) + A(n-1) \\ i=2 &\Rightarrow A(1) + A(n-2) \\ i=3 &\Rightarrow A(2) + A(n-3) \\ &\vdots \\ i=n &\Rightarrow A(n-1) + A(0) \end{aligned}$$

$$= (n+1) + \frac{2}{n} [A(0) + A(1) + \dots + A(n-1)]$$

$$n \cdot A(n) = n \cdot (n+1) + 2 \cdot [A(0) + \dots + A(n-1)]$$

$$-(n-1) \cdot A(n-1) = n \cdot (n-1) + 2 \cdot [A(0) + \dots + A(n-2)]$$

$$n \cdot A(n) - (n-1) \cdot A(n-1) = 2n + 2 \cdot A(n-1)$$

$$\frac{1}{n \cdot (n+1)} \left/ \left[\frac{A(n)}{n+1} - \frac{A(n-1)}{n} = \frac{2}{n+1} \right] \right. *$$

$$t(n) = \sum_{i=2}^n \frac{2}{i+1} = 2 \cdot H(n+1) - 3$$

↘ Harmonic Series

$$i=0 \Rightarrow \frac{2}{2} = 1$$

$$i=1 \Rightarrow \frac{2}{3}$$

$$A(n) = t(n) \cdot (n+1) = 2 \cdot (n+1) \cdot \underbrace{H(n+1)}_{\log(n+1)} - 3 \cdot (n+1)$$

$$\log(n+1) \in \Theta(n \cdot \log n)$$

* Average and best case complexities of quicksort are very close to each other.

A more detailed analysis would show that $A(n) \approx 1.38 \cdot B(n)$

GRAPH SEARCH AND TRAVERSAL ALGORITHMS

Depth-First-Search (DFS) & Breadth-First Search (BFS)

procedure DFS(G, v)

call Visit(v)

Mark [v] = 1

$u = v$

call Next($u, w, found$) // Finds an unvisited neighbour.
while found or (not Empty(Stack))

if found then

call Push(Stack, u)

Mark [w] = 1

call Visit(w)

$u = w$

else

call Pop(Stack, u)

end if

call Next($u, w, found$)

end while

end

Change of Variable : $t(n) = \frac{A(n)}{n+1}$

$$t(n) = t(n-1) + \frac{2}{n+1}$$

Now, it's just a first order recurrence relation.

$$A(0) = 0$$

$$t(0) = \frac{A(0)}{1} = 0$$

⋮

by backward substitution

procedure BFS (G, v)

call Visit (v)

Mark [v] = 1

Enqueue (Queue, v)

while not (Empty (Queue)) do

Dequeue (Queue, u)

for each vertex w adjacent to u do

if Mark [w] = 0 then

call Enqueue (Queue, w)

Mark [w] = 1

Visit (u)

end if

end for

end while

end

BRUTE FORCE AND EXHAUSTIVE SEARCH

Algorithm Design Strategies

Exhaustive search is an important special case of brute force

→ easiest strategy to apply

Ex: Exponentiation

Recall: Computing $a^n \bmod m$ for some large integers is a principal component of a leading encryption algorithm.

$$a^n = \underbrace{a \cdot a \cdot a \cdot \dots \cdot a}_{n \text{ times}}$$

BF is applicable to a wide variety of problems

↳ Ex: Selection Sort
Bubble Sort

Selection Sort:

→ Scan the entire list to find its smallest element

→ Exchange it with the first element.

→ Put the smallest element in its final position in the sorted list

→ Then continue with finding the second smallest etc. [recursively]

* If the list has size n , the list is sorted after $n-1$ passes.

Pseudo code for Selection Sort

```

Algorithm SelectionSort (A[0... n-1])
  for i ← 0 to n-2 do
    min ← i
    for j ← i+1 to n-1 do
      if A[j] < A[min]
        min ← j
      end if
    swap A[i] and A[min]
  end for
end for

```

Analysis:

$$\begin{aligned}
 C(n) &= \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 \\
 &= \sum_{i=0}^{n-2} (n-1-i) \\
 &= (n-1) + (n-2) + \dots + (1) \\
 &= \frac{n \cdot (n-1)}{2} \in \Theta(n^2)
 \end{aligned}$$

Bubble Sort:

- Compare adjacent elements of the list.
- Exchange them if they are out of order.
- By doing this repeatedly, we end up "bubbling up" the largest element to the last position on the list.
- The next pass bubbles up the second longest etc.
- Until after $n-1$ passes the list is sorted

Observation: If a pass through the list makes no exchanges, then the list has been sorted and we can stop the algorithm.

→ This is a new version of the algorithm but its running time is still $\Theta(n^2)$.
(Exercise)

Pseudo code for BubbleSort(A[0... n-1])

```

for i ← 0 to n-2 do
  for j ← 0 to n-2-i do
    if A[j+1] < A[j]
      swap A[j] and A[j+1]
    end if
  end for
end for

```

Analysis:

$$\begin{aligned}
 C(n) &= \sum_{i=0}^{n-2} \sum_{j=0}^{n-2-i} 1 \\
 &= \sum_{i=0}^{n-2} (n-1-i) \\
 &= (n-1) + (n-2) + \dots + (1) \\
 &= \frac{n \cdot (n-1)}{2} \in \Theta(n^2)
 \end{aligned}$$

Ex:

```

28  2  10  5  11  20
  ↔
2  28  10  5  11  20
    ↔
2  10  28  5  11  20
        ↔
2  10  5  28  11  20
            ↔
2  10  5  11  28  20
                ↔
2  10  5  11  20  28
                    ↔
2  5  10  11  20  28
                        |
                        |
                        |
1  1  1  1  1  1

```


Sequential Search and BF String Matching

String Matching Problem: Given a string of n characters called the "text" and a string of m characters called the "pattern", find a substring of the text that matches the pattern.

Ex: patterns "NOT"

NOBODY-NOTICED-HER

Ex: Possible Applications: DNA Sequence Alignment

29.11.2022

Lecture 16

~~Wednesday~~
Tuesday

Pseudocode BF String match ($T[0 \dots n-1]$, $P[0 \dots m-1]$)

for $i \leftarrow 0$ to $n-m$ do

$j \leftarrow 0$

 while $j < m$ and $P[j] = T[i+j]$ do

$j \leftarrow j+1$

 end while

 if $j = m$

 return i

 endif

end for

return -1

Analysis The algorithm may have to make all m comparisons before shifting the pattern and this can happen for each of the $n-m+1$ tries

NONONONONOT
NOT

Then, in the worst case, the algorithm makes $m(n-m+1)$ character comparisons.

Complexity: $O(n \cdot m)$

Closest Pair and Convex Hull Problems by Brute Force

These problems arise in computational geometry and operations research.

Closest Pair Problem: Finding the 2 closest points in a set of points.

Points can represent airplanes, post offices, DB records etc.

2 closest planes are the most probable collision candidates.

Another app: Cluster analysis in statistics

Similarity metric is used for Euclidean distance (finding distances)

* For simplicity we consider the 2-D case

Points $P_i(x_i, y_i)$

Distance btwn. 2 points

P_i and $P_j = d(P_i, P_j)$

$$\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

BF Algorithm for Closest Pair:

- ↳ Compute the distance between each pair of points
- Find a pair with the smallest distance.

Pseudocode for CP:

```

d ← ∞
for i ← 1 to n-1 do
  for j ← i+1 to n do
    d ← min(d, sqrt((xi-xj)2 + (yi-yj)2))
  end for
end for
return d

```

Basic operation

* Even today, computation of sqrt is costly; mainly because it involves dealing with irrational numbers.

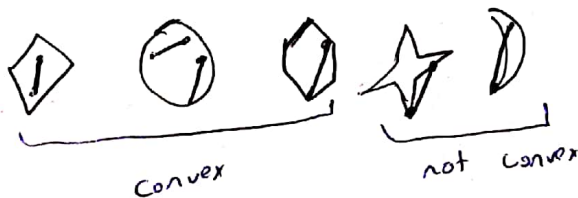
Can it be avoided? Yes!

Only compare $(x_i - x_j)^2 + (y_i - y_j)^2$ since sqrt is strictly increasing. So, the basic operation will be squaring a number.

$$C(n) = \sum_{i=1}^{n-1} \underbrace{\sum_{j=i+1}^n 1}_{n-i \text{ times}} = \sum_{i=1}^{n-1} (n-i) = (n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2} \in \Theta(n^2)$$

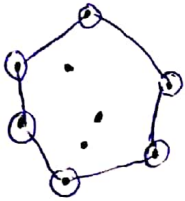
Convex Hull Problem

Convex: A set of points (finite or infinite) in the plane is called convex if for any 2 points p and q in the set, the entire line segment with the endpoints at p and q belongs to the set.



Convex Hull: ~~11/11~~

The convex hull of a set S of points is the smallest convex set containing S . The "smallest" requirement means that the convex hull of S must be a subset of any convex set containing S .



- ⊗ If S is convex, its convex hull is obviously itself 
- ⊗ If S is a set of 2 points, its convex hull is the line segment connecting them.



- ⊗ If S is a set of 3 points that are not on the same line, its convex hull is the triangle with vertices of the given points.

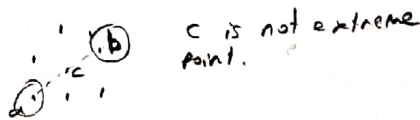
Applications: Convex hulls provide convenient approximations of object shapes and data sets given. In computer animation, replacing objects by their convex hull speeds up collision detection.

- Detecting outliers by some statistical techniques.
- Finding bounding boxes (MS office Visio)
- Visual pattern matching - Detecting license plates

Theorem: The convex hull of any set S of $n > 2$ points not all on the same line is a convex polygon with the vertices at some of the points of S .
 ⇒ we need to find the points that will serve as the extreme points. ↳ we call them "extreme points"

Extreme point: of a convex set is a point of this set that is not a middle point of any line segment with endpoints in the set.

→ we also need to know which pairs of points need to be connected to form the boundary of the convex hull.



c is not extreme point.

→ we can address this issue by listing the extreme points in a clockwise or counter-clockwise order.

Observation: A line segment connecting 2 points P_i and P_j of a set of n points is a part of the convex hull's boundary if and only if all the other points of the set lie on the same side of the straight line through these points.



Algorithm: Repeating this test for every pair of points yields a list of line segments that make up the convex hulls boundary.

* What is the time efficiency of this algorithm?

For each of $\binom{n}{2} = \frac{n \cdot (n-1)}{2}$ pairs of distinct points, we need to check whether all of the remaining points are on the same side.
 $\Downarrow$
 $O(n^3)$

Exhaustive Search

30.11.202

Lecture
Wednesday

Combinational problems deal with combinational objects such as permutations, combinations, and subsets of a given set.

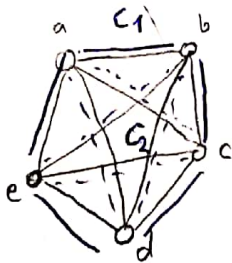
Many of them are optimization problems.

→ ES is simply a BF approach to combinational problems.

↳ It suggests generating each and every element of the problem domain.

Traveling Salesman Problem (TSP)

→ To find the shortest (min-cost) tour through a given set of cities that visits each city exactly once and returns to the original city. (Min-cost Hamiltonian cycle)

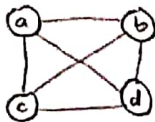


a-b-c-d-e
a-c-e-b-d-a

An ES approach to TSP: Generate all HCs and choose the one with min-cost.

A Ham. Cyc. can also be defined as a sequence of $(n+1)$ adjacent vertices $(v, v, \dots, v, v)$ where the first and the last one are the same.

* Our problem is like ordering around a ring.



a b c d a
a c b d a
a d b c a

$\frac{(4-1)!}{2} = 3$ possibilities
(enumerations)

~~Exhaustive Search~~

Knapsack Problem

n items, profits P_i ,
weights w_i ,
 W : weight limit

The goal is to put items in the knapsack in such a way that the weight limit is not exceeded and the total profit (value) is maximized.

→ Generating all possible subsets, which (2^n subsets), is the ES approach.

⊗ Both TSP and Knapsack are NP-hard problems.

Assignment Problem

n people
 n jobs
1 person/job
 $C[i,j]$: Cost of assigning person i to job j

$$i \begin{bmatrix} & j \\ & \vdots \\ & i,j \\ & \vdots \end{bmatrix}_{n \times n}$$

06.12.2022
Tuesday

The problem boils down to selecting 1 element in each row of the matrix so that all selected elements are in different columns and the total sum of the selected elements is the smallest possible.

→ No obvious solution seems to work here.

Thus, opting for exhaustive search may appear as an unavailable evil.

→ we can describe all feasible solutions to the assignment problem as n -tuples $(j_1, j_2, \dots, j_n)$ in which the i th component indicates the column of the element selected in the i th row.

For instance $(2, 3, 4, 1)$ indicates that

person-1	is assigned to	job-2
person-2	"	job-3
person-3	"	job-4
person-4	"	job-1

So there exist $n!$ candidate solutions.

Therefore, ES is impractical even for very small instances.

However, Hungarian algorithm solves it in polynomial time. ($O(n^3)$ time)

Depth-First-Search and Breadth-First-Search

DFS and BFS systematically process all vertices and edges of a graph. They are fundamental for detecting connectivity and cycle presence.

DFS

↳ Start at an arbitrary vertex and mark it as "visited"

Iteration:

→ Proceed to an adjacent vertex

(Ties are resolved arbitrarily)

→ Continue until a dead end. (a vertex with no adjacent unvisited vertices)

→ At a dead end, back up 1 edge to the vertex it came from & try to continue visiting unvisited vertices from there.

The algorithm eventually halts after backing up to the starting vertex, with the latter being a dead end.

* Accompanying a DFS traversal by constructing the DFS forest.

Pseudo code

visits recursively all unvisited vertices connected to a vertex v by a path and numbers them in the order they are encountered via a global variable "count"

Algorithm DFS(G)

count $\leftarrow 0$

for each vertex v in V do

if v is marked with 0

DFS(v)

count $\leftarrow$ count + 1 // Node v with count

for each vertex w in v adjacent to v do

DFS(w)

How efficient is DFS?

* For adjacency matrix representation: $O(|V|^2)$
since there exist $|V|^2$ entries

* For adjacency list representation: $O(|V| + |E|)$

* Checking connectivity using DFS: when the algorithm halts, check whether all vertices have been visited.

We can also use DFS to identify connected components

* Checking for a cycle presence using DFS: If the DFS forest (notice that the DFS forest is not actually a forest) does not have back edges, the graph is acyclic.

Breadth-First-Search (BFS)

→ Proceeds in a concentric manner by visiting first all vertices adjacent to a starting vertex, then others etc.

⊛ It is convenient to use queue as a data structure.

On each iteration

↳ Identify all unvisited vertices adjacent to that vertex

⇒ Mark them as visited

⇒ Add them to the queue and then remove the front vertex.

Pseudocode for BFS (G)

0 is a mark of being "unvisited"

count $\leftarrow$ 0

for each vertex v in V do

if v is unmarked

bfs(v)

bfs(v) // visits all ^{unvisited} vertices connected to v

count $\leftarrow$ count + 1

Initialize a queue with v and mark v with count

While the queue is not empty do

for each vertex w in V adjacent to the front vertex do

if w is marked with 0

count $\leftarrow$ count + 1, mark w with count

add w to the queue

remove the front vertex from the queue

⊛ BFS has the same efficiency as DFS.
Like DFS, BFS can be used for connectivity & acyclicity

Decrease and Conquer

→ Decrease and conquer technique is based on exploiting the relationship between a solution to a given instance of a problem and a solution to its smaller instances.

① Decrease-by-a-constant: The size of an instance is reduced by the same constant on each iteration of the algorithm. Typically, this constant equals 1.

Ex:

① Exponentiation: Computing $a^n = a^{n-1} * a$

② Insertion sort

③ Factorial

② Decrease-by-a-constant-factor: Suggests reducing a problem instance by the same constant factor on each iteration.
In most applications, this factor equals 2

Ex:

① Again exponentiation

$$a^n = \begin{cases} (a^{n/2})^2 & \text{if } n \text{ is even} \\ (a^{(n-1)/2})^2 * a & \text{if } n \text{ is odd} \end{cases}$$

② Binary search

Variable Size Decrease

The size reduction pattern varies from one iteration of the algorithm to another.

Ex: Euclid's algorithm for computing greatest common divisor (GCD):

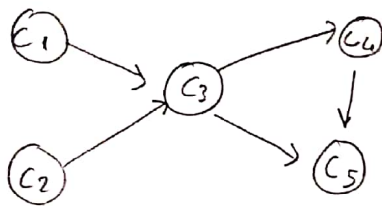
$$\gcd(m, n) = \gcd(n, m \bmod n)$$

↳ The value decreases neither by a constant nor by a constant factor

has a variety of applications involving prerequisite-related tasks (like courses with prerequisites)

Problem: In CS, a topological sort or topological ordering of a digraph is a linear ordering of its vertices such that for every directed edge $u \rightarrow v$ from vertex u to vertex v , u comes before v in the ordering.

Ex:



A possible topological sorting: C_1, C_2, C_3, C_4, C_5

Some other applications: ① A large project in construction, research or SW development that involves a multitude of interrelated tasks with known prerequisites.

Ex: Critical path method in SW engineering

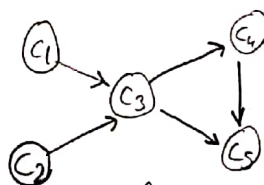
- ② Instruction scheduling in program compilation
- ③ Cell evaluation ordering in spreadsheet formulas
- ④ Resolving symbol dependencies in linkers

Question: When does topological sorting not have a feasible solution:

Answer: When the graph has a directed cycle.

If the graph is a DAG (Directed Acyclic Graph), then it has a feasible solution.

Algorithm-1: Perform a DFS traversal and note the order in which vertices become dead ends (without outgoing edges).



C_5, C_4, C_3, C_1, C_2 → Reverse order

C_2, C_1, C_3, C_4, C_5

C_5, C_4, C_3, C_2, C_1 → Reverse order

C_1, C_2, C_3, C_4, C_5

Algorithm-2: Direct implementation of the decrease-and-conquer technique:
→ Repeatedly identify in the remaining graph a source (a vertex with no incoming edges), note it down and delete it along with all edges outgoing from it

Algorithms for generating combinatorial objects

Some types of combinatorial: Permutations, combinations, subsets of a given set

- * Combinatorial objects are studied in a branch of discrete mathematics called combinatorics
- * The # of combinatorial objects typically grows exponentially or even faster as a function of the problem size.

Our goal: Generating them, not counting them.

Generating
permutations

Read it from your book.

13.12.2022

Tuesday

Generating Subsets

Recall the knapsack problem

Power set of n : It has cardinality 2^n

The set of all subsets

Decrease-by-1 idea: All subsets of $A = \{a_1, a_2, \dots, a_n\}$ can be divided into 2 groups:

- (1) sets that do not have a_n
 - (2) sets that do have a_n
- Once we have a list of all subsets of $\{a_1, a_2, \dots, a_{n-1}\}$, we can get all subsets of $\{a_1, \dots, a_n\}$ by adding $\{a_n\}$ to each subset.

n	
step 0	$\emptyset$
step 1	$\emptyset, \{a_1\}$
step 2	$\emptyset, \{a_1\}, \{a_2\}, \{a_1, a_2\}$
step 3	$\emptyset, \{a_1\}, \{a_2\}, \{a_1, a_2\}, \{a_3\}, \{a_1, a_3\}, \{a_2, a_3\}, \{a_1, a_2, a_3\}$

* All 2^n bit strings $b_1, b_2, \dots, b_n$ of length n can be generated in the same way.

Bit string	000	001	010	...	011	...	111
Subsets	$\emptyset$	$\{a_1\}$	$\{a_2\}$		$\{a_1, a_2\}$		$\{a_2, b_3, a_3\}$

* A challenging question is whether there exist a "minimal change algorithm" for generating bit strings so that every one of them differs from its immediate neighbor by only a single bit

Ex 000 } Cycle $\Rightarrow$ Such a sequence of bits
001 } is called "binary reflected"
010 }
...

BRG(n)

Algorithm Binary-Reflected-Graycode// Input: A positive integer n // Output: List of all bit strings of length n comprising the Gray Code.If $n=1$ Make list L containing bit strings 0 and 1 in this order.

Else

Generate list $L1$ of bit strings of size $n-1$ by calling $BRGC(n-1)$ Copy list $L1$ to list $L2$ in reverse orderAdd 0 in front of each bit string in $L1$ Add 1 " " " " " " in $L2$ Append $L2$ to $L1$ to get List L Return L * Note that $BRGC$ is cycle
$$L1 : 0, 1 \xrightarrow{\text{Add 1}} 00, 01$$

$$L2 : 1, 0 \xrightarrow{\text{Add 1}} 11, 10$$

000	100
001	101
010	110
011	111
100	100

Fake Coin ProblemAmong n identical looking coins, one is false. With a balance scale, we can compare any 2 sets of coins. Continue with dividing by 2.Divide n coins into 2 piles of $\lfloor \frac{n}{2} \rfloor$ coins each, leaving 1 extra coin aside if n is odd. Resulting recursion formula;

$$W(n) = W(\lfloor \frac{n}{2} \rfloor) + 1 \text{ for } n > 1$$

$$W(1) = 0$$

↓

* This recursion is identical to the one for the worst case number of comparisons for binary search.

* The interesting point here is that the above algorithm is not the most efficient solution.

It would be more efficient to divide the coins into 3 piles of about $\frac{n}{3}$ coins each

Russian Peasant Multiplication

 n and $m \Rightarrow$ positive integersRecursive Formula $\Rightarrow$ If n is even $\Rightarrow n \cdot m = \frac{n}{2} \cdot 2m$ If n is odd $\Rightarrow n \cdot m = \frac{n-1}{2} \cdot 2m + m$ ⊗ This way, we can compute $n \cdot m$ either recursively or iteratively.

Ex :

n	m	
70	85	
35	170	
17	340	(+170)
8	680	(+340)
4	1360	<u> </u>
2	2720	510
1	5440	5950

⊗ According to Western visitors, this method was used widely by Russian peasants in the 19th century.

⊗ In fact, the method was known to Egyptian mathematicians as early as 1650 BC.

⊗ This method leads to very fast HW implementations since doubling and halving of binary numbers can be performed using binary shifts.

Variable-Size-Decrease Algorithms

 $\hookrightarrow$ The size reduction pattern varies from one iteration to another

Ex: Euclid's algorithm for computing GCD

Computing A Median And The Selection Problem

Selection Problem: Finding the k^{th} smallest element in a list of n numbers.This number is called the k^{th} order statistic.When $k = \lfloor \frac{n}{2} \rfloor$, this number is called the median.Obviously, we can find it by sorting the list and then finding the k^{th} smallestBut wouldn't it be an overkill because we don't need the entire list, but just the k^{th} smallest element? $\Downarrow$
 $O(n \log n)$
Sorting

Can we do better?

⊗ Partitioning a given list around some value p :

Rearrangement of the elements such that:

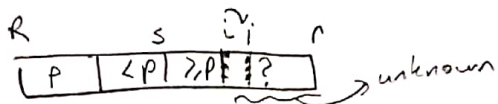
① Left part contains elements $< p$ ② p : pivot element③ Right part contains elements $> p$

Lomuto Partitioning

A subarray $A[l, \dots, r]$ ($0 \leq l \leq r \leq n-1$) composed of 3 continuous segments:

- ① A segment with elements known to be smaller than P
- ② A " " " " " " " $\geq P$
- ③ " " " " yet to be compared to P

→ The segments can be empty. This is always the case for the first 2 segments before the algorithm starts.



Starting with $i = l + 1$, the algorithm scans the subarray $A[l, \dots, r]$ left to right, maintaining this structure until a partition is achieved.

On each iteration

- ↳ Compare the first element in the unknown segment (pointed to by i) by the pivot P
- ↳ If $A[i] \geq P$: Increment i to expand the segment with elements $\geq P$ while shrinking the unprocessed segment
- ↳ If $A[i] < P$: The segment with elements $< P$ needs to be expanded. This is done by:
 - ① Incrementing s (the index of the last element in the first segment)
 - ② Swapping $A[i]$ and $A[s]$
 - ③ Incrementing i to point to the new first element of the shrunk unprocessed element

Pseudocode Lomuto Partitioning ($A[l, \dots, r]$)

$p \leftarrow A[l]$

$s \leftarrow l$

for $i \leftarrow l + 1$ to r do

if $A[i] < p$

$s \leftarrow s + 1$

swap ($A[s], A[i]$)

endif

end for

swap ($A[l], A[s]$)

return s

⇒ After no unprocessed elements remain, the algorithm swaps the pivot with $A[s]$ to achieve the partition being sought

* How can we take advantage of a list partition to find the k^{th} smallest in it?

Indices start with 0

s = Split position of the partition = index of the array's element occupied by the pivot after partitions

- ① If $s = k-1$, pivot p is the k^{th} smallest element.
- ② If $s > k-1$, k^{th} smallest element is the k^{th} smallest of the left part
- ③ If $s < k-1$, k^{th} " " " " $(k-s)^{\text{th}}$ smallest element in the right part

Pseudo code for Quick Select ($A[l \dots r], k$)

```

s ← Lomuto partition ( $A[l \dots r]$ ) // or another partitioning algorithm
if  $s = k-1$  return  $A[s]$ 
else if  $s > k-1$ 
    QuickSelect ( $A[l \dots s-1], k$ )
else
    QuickSelect ( $A[s+1 \dots r], k-s$ )
end

```

How efficient is Quick Select?

Best Case: Partitioning an n -element array always requires $n-1$ key comparisons. If we don't require more iterations, this is the best case

$$C_{\text{best}}(n) = n-1 \in \Theta(n)$$

Worst Case: An extremely unbalanced partition (with 1 part empty and the other part $(n-1)$ elements)

↓

This can happen on each of the $(n-1)$ iterations

$$C_{\text{worst}}(n) = (n-1) + (n-2) + \dots + 1 = \frac{n \cdot (n-1)}{2} \in \Theta(n^2)$$

- * A careful mathematical analysis shows that the average case is linear.
- * A more sophisticated way of choosing a pivot guarantees linear time even in the worst case (proved in 1973)
- * Notice that the partitioning based algorithm identifies the k smallest and $n-k$ largest elements, not just the value of the k^{th} smallest element

Divide & Conquer

→ Also famous for its catchy name

- ① Divide the problem into several sub problems of the same type, ideally about equal size.
- ② Solve the sub problems. (typically recursively)
- ③ Combine the solutions to the subproblems to get a solution to the original problem.

Spend only linear time for the initial division and final recombining.

- ⊛ In many settings where divide and conquer is applied, the natural brute force algorithm may already be polynomial time. In this case, D&C strategy may serve to reduce the running time to a lower polynomial.

Ex: Computing the sum of n numbers

- Compute the sum of the first $\lfloor \frac{n}{2} \rfloor$ numbers
- Compute the sum of the second $\lfloor \frac{n}{2} \rfloor$ numbers
- Add their values to get the sum in question.

Ex: The natural BF approach for closest pair problem has complexity $\Theta(n^2)$

D&C approach yields $O(n \log n)$

⊛ Typical recurrence for D&C:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

→ a function that accounts for dividing the instance of size n into instances of size $\frac{n}{b}$ & combining their solutions.

⊛ If $a=1$, D&C covers decrease-by-a-constant factor algorithms.

Merge Sort

Divide the array into 2 halves, sort each one recursively and merge them to a single sorted one.

Algorithm MergeSort($A[0 \dots n-1]$)

if $n > 1$

copy $A[0, \dots, \lfloor \frac{n}{2} \rfloor - 1]$ to $B[0, \dots, \lfloor \frac{n}{2} \rfloor - 1]$
 copy $A[\lfloor \frac{n}{2} \rfloor, \dots, n-1]$ to $C[0, \dots, \lfloor \frac{n}{2} \rfloor - 1]$
 MergeSort($B[0, \dots, \lfloor \frac{n}{2} \rfloor - 1]$)
 MergeSort($C[0, \dots, \lfloor \frac{n}{2} \rfloor - 1]$)
 Merge(B, C, A)

The merging of 2 sorted arrays can be done as follows:

- ⊛ 2 pointers are initialized to point to the first elements of the arrays.
- ⊛ Compare the elements pointed to, copy the smaller one to the new array & increment the index of the smaller element

Algorithm Merge ($B[0, \dots, p-1]$, $C[0, \dots, q-1]$, $A[0, \dots, p+q-1]$)

$i \leftarrow 0$; $j \leftarrow 0$; $k \leftarrow 0$

while $i < p$ and $j < q$ do

if $B[i] \leq C[j]$

$A[k] \leftarrow B[i]$

$i \leftarrow i+1$

else

$A[k] \leftarrow C[j]$

$j \leftarrow j+1$

end

$k \leftarrow k+1$

end

if $i = p$

copy $C[j, \dots, q-1]$ to $A[k, \dots, p+q-1]$

else

copy $B[i, \dots, p-1]$ to $A[k, \dots, p+q-1]$

$$C(n) = 2 \cdot C\left(\frac{n}{2}\right) + C_{\text{merge}}(n)$$

At each step, exactly 1 comparison is needed. In the worst case, neither of the 2 arrays become empty before the other one contains just 1 element.

Hence, $C_{\text{merge}}(n) = n-1$ and we have

$$C_{\text{worst}}(n) = 2 C_{\text{worst}}\left(\frac{n}{2}\right) + n-2$$

$$C_{\text{avg-worst}}(n) \in \Theta(n \log n)$$

⊗ what if n is odd:

$$T(n) = T\left(\lceil \frac{n}{2} \rceil\right) + T\left(\lfloor \frac{n}{2} \rfloor\right) + Cn \rightarrow n-1$$

⊗ Asymptotic bounds do not change

⊗ Multi-way Mergesort: Divide a list to be sorted in more than 2 parts, sort each one recursively, and then merge them together

$$T_{\text{3-way merge sort}} = 3 \cdot T\left(\frac{n}{3}\right) + n-1 \rightarrow f(n) \in \Theta(n)$$

$$a=3$$

$$b=3$$

$$f(n)$$

$$\Downarrow$$

$$d=1$$

$$a=3=3' \Rightarrow T(n) \in \Theta(n \log n)$$

- * A noteworthy advantage of mergesort over quicksort is its stability.
- * The principal shortcoming of mergesort is the linear amount of extra storage the algorithm requires.

Quicksort

Unlike mergesort, which divides its input elements according to their position, quicksort divides them according to their value.

Partition $\Rightarrow$ A partition is an arrangement of the array's elements so that all elements to the left of some element $A[s]$ are less than $A[s]$ and all elements to the right of some element $A[s]$ are greater than $A[s]$.

$\rightarrow$ In mergesort, the division is obvious and the entire work happens in combining their solutions.

$\rightarrow$ In quicksort, the entire work happens in the division stage, with no work required to combine the solutions to the subproblems.

Algorithm Quicksort ($A[l \dots r]$)

if $l < r$

$s \leftarrow \text{Partition}(A[l \dots r])$

Quicksort($A[l \dots s-1]$)

Quicksort($A[s+1 \dots r]$)

// s is a split position

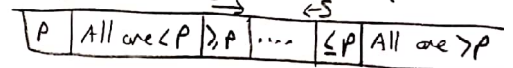
$\Rightarrow$ As a partitioning algorithm, we can certainly use Lomuto partition or a more sophisticated method by Hoare as follows

$\rightarrow$ We start by selecting a pivot

For now, do the simplest selection: choose $p = A[l]$

* Unlike the Lomuto algorithm, now scan the subarray from both ends.

comparing the subarray's elements with the pivot



i : Denotes the left-to-right scan and starts with the second element

This scan stops upon encountering the first element $>$ pivot

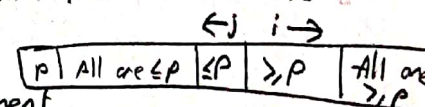
j : Denotes the right-to-left scan and starts with the last element.

This scan stops upon encountering the first element $\leq$ pivot.

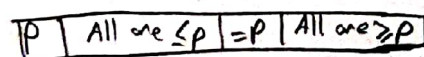
After both scans stop, 3 situations may arise:

① if i & j have not crossed (i.e. $i < j$), we simply exchange $A[i]$ and $A[j]$ and resume the scans by incrementing i & decrementing j .

② If they have crossed over (i.e. $i > j$), we will have partitioned the subarray after exchanging the pivot with $A[j]$:



③ If i and j stop while pointing to the same element (i.e. $i = j$), then the value they are pointing to must be equal to p . Thus, we already have the subarray partitioned with the split position $s = i = j$



Algorithm Hoare Partition ($A[l \dots n]$)

$p \leftarrow A[l]$ // pivot

$i \leftarrow l$

$j \leftarrow i + 1$

repeat

repeat $i \leftarrow i + 1$ until $A[i] \geq p$

repeat $j \leftarrow j - 1$ until $A[j] \leq p$

swap($A[i], A[j]$)

until $i \geq j$

swap($A[i], A[j]$) //undo the last swap when $i > j$

swap($A[l], A[j]$)

return j

21.12.2022
Wednesday
Lecture 23

Efficiency: The number of key comparisons made before a partition is achieved.

$$= \begin{cases} n+1 & \text{if the scanning indices cross over} \\ n & \text{if they coincide} \end{cases}$$

→ If all splits happen in the middle of the corresponding subarrays, we have the best case.

$$C_{\text{best}}(n) = 2 \cdot C_{\text{best}}\left(\frac{n}{2}\right) + n \quad \text{for } n > 1, \quad C_{\text{best}}(1) = 0$$

⇔

According to Master Theorem, $C_{\text{best}}(n) \in \Theta(n \log n)$

Solving exactly for $n = 2^k$ yields: $C_{\text{best}}(n) \in \Theta(n \log n)$

Worst-Case: In the worst case, all splits are skewed to the extreme:

1 subarray is empty

The other subarray has size just 1 less than the size of the subarray being partitioned

⇔

This situation occurs for inputs that are already sorted.

$A[0] \ A[1] \ \dots \ A[n-1]$

↗ ↘

$(n+1)$ comparisons in iteration 1

(n) comparisons in iteration 2

⋮

3

..

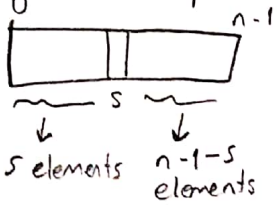
..

..

$$= \frac{(n+1)(n+2)}{2} - 3 \in \Theta(n^2)$$

Average Case: A partition can happen in any position s ($0 \leq s \leq n-1$) after $n+1$ comparisons are made to achieve the partition.

* Assuming that the partition split can happen in each position s with the same probability $\frac{1}{n}$, we get the following recurrence:



$$C_{avg}(n) = \frac{1}{n} \sum_{s=0}^{n-1} [(n+1) + C_{avg}(s) + C_{avg}(n-1-s)] \quad \text{for } n > 1$$

$$C_{avg}(0) = 0$$

$$C_{avg}(n) = 0$$

Full history recurrence

* Better pivot selection methods

* Switching to insertion sort on very small subarrays (5-15 elements)

* Quicksort is not stable

$$C_{avg}(n) \in \Theta(n \log n)$$

Counting Inversions



2 indices $i < j$ form an inversion if $a_i > a_j$ (out of order)

Ex: Collaborative filtering:

They try to match your preferences (for books, movies, etc.) with those of other people out on the Internet.

Identifying people with "similar" interest

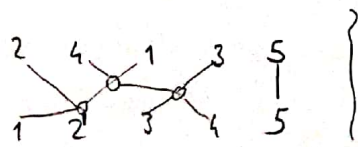
→ Another app: Meta-search tools

Execute the same query on many different search engines & then synthesize the results by comparing the rankings.

A core issue: The problem of comparing two rankings

What is a good way to measure how similar 2 people's rankings are?

* Given a sequence of n distinct numbers $a_1, \dots, a_n$ determine the number of inversions in the sequence.



Each crossing pair of line segments corresponds to 1 pair that is in the opposite order in the 2 lists.

In ascending order $\Rightarrow 0$ inversions

In descending order $\Rightarrow \frac{n \cdot (n-1)}{2}$ inversions

Designing & Analyzing The Algorithm

Simplest algorithm (ES) = Check each pair $\Rightarrow O(n^2)$ time

Now we'll show a D & C algorithm to count them in $O(n \log n)$ time.

Such an algorithm should behave without looking at each inversion individually.

→ Set $m = \lceil \frac{n}{2} \rceil$ and divide the list into the 2 pieces $a_1, \dots, a_m$ and

→ Then count the number of inversions (a_i, a_j) where $a_1, \dots, a_m$ and $a_{m+1}, \dots, a_n$ the 2 numbers belong to different halves.

We must do this part in $O(n)$ time.

* These inversions have a nice form: They are precisely the pairs (a_i, a_j) , where a_i is in the first half and a_j is in the second half, and $a_i > a_j$.

* We will make the algorithm recursively sort the numbers in the 2 halves as well

→ After sorting, we have 2 sorted lists A and B, containing the first and second halves respectively. We want to produce a single sorted list C from their union, while also counting the number of pairs (a, b) with $a \in A, b \in B$ and $a > b$

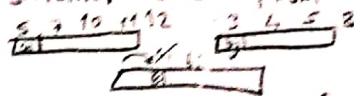
27.12.2022

Tuesday
Lecture 24

→ In each step, have a current pointer into each list

Suppose that these pointers are currently at elements a_i and b_j

→ In each step, compare the elements a_i and b_j being pointed to in each list, remove the smaller one from the list and append it to the end of list C.

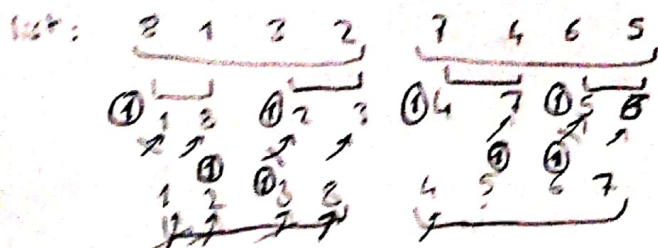


→ Every time the element a_i is appended to C, no new inversions are encountered since a_i is smaller than everything left in list B.

→ On the other hand, if b_j is appended to list C, then it is smaller than all the remaining items in A and comes after all of them. So, we increase our count of the number of inversions by the number of remaining elements in A.

This way, in constant time, we have accounted for a potentially huge number of inversions.

Ex:



1 2 3 4 5 6 7 8

Pseudo code :

Merge-and-count(A, B)

- Maintain a current pointer into each list, initialized to point to the front elements
- Maintain a variable count for the number of inversions, initialized to 0.
- While both lists are nonempty:
 - Let a_i and b_j be the elements pointed to by the current pointer.
 - Append the smaller of these two to the output list
 - If b_j is the smaller element then
 - Increment count by the number of elements remaining in A
 - End if
 - Advance the current pointer in the list from which the smaller element was selected
- Once the list is empty, append the remainder of the other list to the output.
- Return count and the merged list

⊛ Running time of Merge-and-Count

- Each iteration of the while loop takes constant time
- In each iteration, we add some element to the output that will never be seen again.

Thus, the number of iterations can be at most the sum of the initial lengths of A & B, so the total running time is $O(n)$

Pseudo Code for Sort-and-Count(L)

If the list has 1 elements, then 0 inversions.

Else

Divide the list into 2 halves;

A contains the first $\lceil \frac{n}{2} \rceil$ elements

B contains the remaining $\lfloor \frac{n}{2} \rfloor$ elements

$(r_A, A) = \text{Sort-and-count}(A)$

$(r_B, B) = \text{Sort-and-count}(B)$

$(r, L) = \text{Merge-and-count}(A, B)$

return $r \leftarrow r_A + r_B + r$ and the sorted List L

Since Merge-and-count takes $O(n)$ time, the entire running time satisfies the recurrence

$$T(n) \leq 2 \cdot T\left(\frac{n}{2}\right) + cn$$

⇓

$$T(n) \in O(n \log n)$$

Finding The Closest Pair of Points

The set of points = $P = \{P_1, \dots, P_n\}$, where P_i has coordinates (x_i, y_i)

$d(P_i, P_j)$: The euclidean distance between two points

Goal: To find a pair of points P_i, P_j that minimizes $d(P_i, P_j)$

→ Assume that no 2 points have the same x -coordinate or y -coordinate

→ Divide the area into 2 halves

Find the closest pair in the left half } Then use this information to
Find the closest pair in the right half } get the overall solution in linear time (combining step)

Combining Step: The distances that have not been considered by either of the recursive calls are precisely those points that occur between a point in the left half and a point in the right half

↓

There exists $\Omega(n^2)$ such distances and we need to find the smallest one in $O(n)$ time,

On a set $P' \subseteq P$, every recursive call begins with 2 lists:

→ A list P'_x in which all points in P' have been sorted by increasing x -coordinate.

→ A list P'_y in which all points in P' have been sorted by increasing y -coordinate.

$Q \triangleq$ The set of points in the first $\lceil \frac{n}{2} \rceil$ positions of the list P_x (the "left half")

$R \triangleq$ The set of points in the final $\lfloor \frac{n}{2} \rfloor$ positions of the list P_x (the "right half")

→ By a single pass through each of P_x and P_y (in $O(n)$ time), we can create the following 4 lists:

① $Q_x \triangleq$ points in Q sorted by increasing x -coordinate

② $Q_y \triangleq$ points in Q sorted by " " y - " "

③ $R_x \triangleq$ " " R " " " x - " "

④ $R_y \triangleq$ " " R " " " y - " "

For each entry (point), we record its position in both lists it belongs to

Combining The Solution:

q_0^* and q_1^* : Closest pair of points in Q

r_0^* and r_1^* : Closest pair of points in R

$$\text{Let } \delta = \min \{d(q_0^*, q_1^*), d(r_0^*, r_1^*)\}$$

→ Are there points $q \in Q$ and $r \in R$ for which $d(q, r) < \delta$

$x^* \triangleq$ x-coordinate of the rightmost point in Q

Lemma: If there exists $q \in Q$ and $r \in R$ for which $d(q, r) < \delta$, then each of q and r lies within a distance δ of L .

Proof: Suppose that such q and r exist. We write $q = (q_x, q_y)$ and $r = (r_x, r_y)$.

By definition of x^* , we know that $q_x \leq x^* \leq r_x$.

Then we have:

$$x^* - q_x \leq r_x - q_x \leq d(q, r) < \delta$$

$$r_x - x^* \leq r_x - q_x \leq d(q, r) < \delta$$

So, each of q and r has an x-coordinate within δ of x^* and lies within distance δ of the line L .

28.12.2022

Wednesday

Lecture 25

So, if we want to find a close q and r , we can restrict our search to the narrow band consisting only of points in P within δ of L .

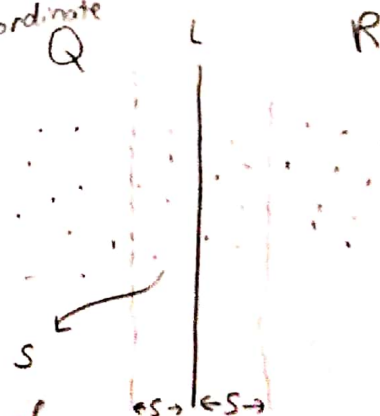
Let $S \subseteq P$ denote this set (set of points in P within δ of L .)

S_y : Set of points in S sorted by increasing y-coordinate

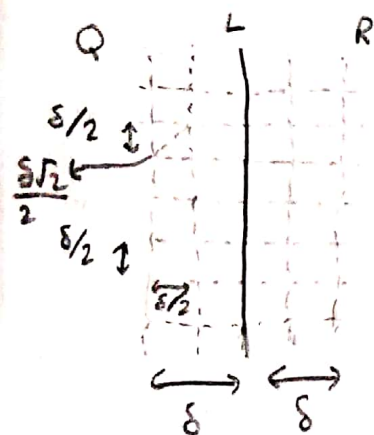
By a single pass through the list P_y , we can construct S_y in $O(n)$ time.

A restatement of the previous lemma: There exist $q \in Q$ and $r \in R$ for which $d(q, r) < \delta$ iff there exist $s, s' \in S$ for which $d(s, s') < \delta$

Lemma: If $s, s' \in S$ have the property that $d(s, s') < \delta$, then s and s' are within 15 positions of each other in the sorted list S_y .



Proof: Consider the subsets Z of the plane consisting of all points within distance δ of L . We partition Z into boxes as in here.



We first prove that each box contains at most 1 point of S . Suppose that 2 points of S lie in the same box. Since all points in this box lie on the same side of the line L , these 2 points either both belong to Q or both belong to R . However, any 2 points in the same box are within distance $\frac{\delta\sqrt{2}}{2} < \delta$, which contradicts with our definition of δ as the minimum distance between any pair of points in Q or in R . Thus, each box contains at most 1 point of S .

Now suppose that $s, s' \in S$ have the property that $d(s, s') < \delta$ and that they are at least 16 positions apart in S_y . Assume without loss of generality that s has the smaller y -coordinate. Then, since there can be at most 1 point per box, there exist at least 3 rows of Z lying between s and s' .

However, any 2 points in Z separated by at least 3 rows must be a distance of at least $\frac{3\delta}{2}$ apart, a contradiction.

This value of 15 can be reduced. However, for our purposes, important thing is that it is an absolute constant.

* We make 1 pass through S_y and for each $s \in S_y$, we compute its distance to each of the next 15 points in S_y . This way, we can compute the distance of each pair of points in S that are at distance less than δ from each other.

Dynamic Programming

DP is a technique for solving problems with overlapping subproblems.

Typically, these subproblems arise from a recurrence relating a given problem's solution to solutions of its smaller subproblems.

⇒ Rather than solving subproblems again and again, solve each of the smaller subproblems only once and record the results in a table from which a solution to the original problem can then be obtained.

Fibonacci Numbers

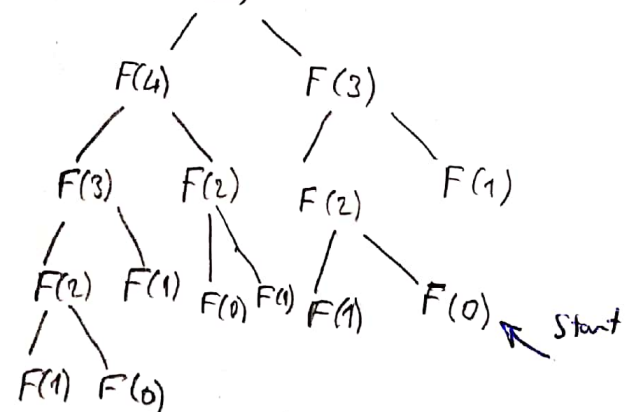
Recurrence Relation

$$F(0) = 0$$

$$F(1) = 1$$

$$F(n) = F(n-1) + F(n-2) \quad \text{for } n > 1$$

$$F(5)$$



DP Table

F	0	1	2	3	4	5
	0	1	2	3	4	5

If we use this recurrence directly to compute the n th Fibonacci number, as we see on this figure, we would have to recompute the same values of this function many times.



Instead, we can simply fill all elements of a 1D array by using the initial conditions and fill the rest.

Algorithm $Fib(n)$

$$F(0) \leftarrow 0$$

$$F(1) \leftarrow 1$$

for $i \leftarrow 2$ to n do

$$F(i) \leftarrow F(i-1) + F(i-2)$$

end for

return $F(n)$

DP → In general leads to space-time trade off since it uses more space to save from time.

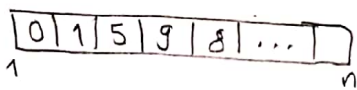
Crucial Step: Deriving the recurrence relation

Principle of Optimality: An optimal solution to any instance of an optimization problem is compared of optimal solutions to its subinstances. For instance, this principle does not exist in finding Longest-Path problem but it exists in Shortest-Path Problem.

Some Basic Examples

Coin Row Problem: There is a row of coins whose values are some positive integers $c_1, c_2, \dots, c_n$ not necessarily distinct.

The goal is to pickup the maximum amount of money subject to the constraint that no 2 coins adjacent in the row can be picked up.



Let $F(n)$ = Maximum amount that can be picked up from a row of n coins.

Partition the allowed coin selections into 2 groups.

① Those that include the last coin

② Those that do not include the last coin

→ The largest amount we can get from here = $C_n + F(n-2)$

→ The largest amount we can get from here : $F(n-1)$

Hence, we have the following recurrence:

$$F(n) = \max \{ C_n + F(n-2), F(n-1) \}$$

$$F(1) = C_1 \quad \text{for } n \geq 2$$

$$F(0) = 0$$

Algorithm Coin Row ($C[1 \dots n]$)

$F[0] \leftarrow 0;$

$F[1] \leftarrow C[1]$

for $i \leftarrow 2$ to n do

$F[i] \leftarrow \max (C[i] + F[i-2], F[i-1])$

end for

return $F[n]$

Example: Given coin row of denominators. 5, 1, 2, 10, 6, 2

Index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5	7	15	15	17

DP Table

This algorithm clearly takes $\Theta(n)$ time and $\Theta(n)$ space

Change Making Problem

→ DP algorithm for the general case, assuming availability of unlimited quantities of coins for each of the m denominations.

$$d_1 < d_2 < \dots < d_m \text{ where } d_1 = 1$$

Let $F(n)$ = Minimum # of coins whose value add up to n ($= \text{OPT}(n)$)
 $F(0) = 0$

⊗ The amount n can only be obtained by adding 1 coin of denomination d_j to the amount $n - d_j$ for $j = 1, 2, \dots, m$ such that $n \geq d_j$.

Hence, we can consider all such denominations and select the one minimizing $F(n - d_j) + 1$. So, we have the following recurrence.

$$F(n) = \min_{j: n \geq d_j} \{ F(n - d_j) \} + 1$$

↓

Again, we can fill a 1-row table. However, this time, computing a table entry requires finding the minimum up to n numbers

Algorithm Change Making ($D[1 \dots m], n$)

$F[0] \leftarrow 0$

for $i \leftarrow 1$ to n do

temp $\leftarrow \infty$

$j \leftarrow 1$

while $j \leq m$ and $D[j] \leq i$ do

temp $\leftarrow \min(F[i - D[j]], \text{temp})$

$j \leftarrow j + 1$

end while

$F[i] \leftarrow \text{temp} + 1$

return $F[n]$

Application of the algorithm for $n=6$ and denominations 1, 3, 4;

n	0	1	2	3	4	5	6
F	0	1	2	1	1	2	2

$$F[1] = \min(F[1-1]) + 1$$

$$F[3] = \min \left(\begin{matrix} F[3-1] & F[3-3] \\ 2 & 0 \end{matrix} \right) + 1$$

$$F[4] = \min \left(\begin{matrix} F[4-1] & F[4-3] & F[4-4] \\ & & \rightarrow 0 \end{matrix} \right) + 1$$

$$F[5] = \min \left(\begin{matrix} F[5-1] & F[5-3] & F[5-4] \\ & & \rightarrow 1 \end{matrix} \right) + 1$$

$$F[6] = \min \left(\begin{matrix} F[6-1] & F[6-3] & F[6-4] \\ & \rightarrow 3 & \rightarrow 0+1 \end{matrix} \right) + 1$$

⊗ Time efficiency of the algorithm: $O(n \cdot m)$

There exist m denominations and n entries.

⊗ Space efficiency $O(n)$

m and n comparisons are made for each entry

Knapsack Problems

Given n items with known weights $w_1, \dots, w_n$ and values $v_1, \dots, v_n$ and a knapsack capacity W .

→ For a DP algorithm, first ^{numeric value.} derive the recurrence relation

* Consider an instance with the first i items $1 \leq i \leq n$ with weights $w_1, \dots, w_i$ and values $v_1, \dots, v_i$ and knapsack capacity j $[1 \leq j \leq W]$

Let $F(i, j)$ = The value of an optimum solution to this instance

= The value of the most valuable subset of the first i items that fit into the knapsack of capacity j .

We can divide all feasible solutions to this instance into 2 categories:

① The subsets that do not include the i th item: Optimum solution is $F(i-1, j)$

② The subsets that do include the i th item (hence, $j - w_i > 0$)

Optimum solution is

$$v_i + F(i-1, j-w_i)$$

Of course, if the i th item does not fit into the knapsack, $F(i, j) = F(i-1, j)$

Hence, we have the following recurrence:

(Case 1)

$$F(i, j) = \begin{cases} \max(F(i-1, j), v_i + F(i-1, j-w_i)) & \text{if } j-w_i > 0 \\ F(i-1, j) & \text{if } j-w_i < 0 \end{cases}$$

Initial conditions:

$$F(0, j) = 0 \quad \text{for } j > 0$$

$$F(i, 0) = 0 \quad \text{for } i > 0$$

* Our goal is to find $F(n, W)$;

Make a 2-D table:

DP Table

	0	...	$j-w_i$	...	j	...	W
0	0		0		0		0
...							
$i-1$	0		$F(i-1, j-w_i)$		$F(i-1, j)$		
...							
i	0						
...							
n	0						

Goal

Example:

Item	Weight	Value
1	2	\$12
2	1	\$10
3	3	\$20
4	2	\$15

Capacity = $W = 5$

We want to find $F(4, 5)$

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25	30	37

Annotations:

- capacity
- $F(3, 4) = \max(F(2, 4), 20 + F(2, 1))$
- $F(3, 2) = F(2, 2)$
- $j = w_i < 0$
- $F(3, 3) = \max(F(2, 3), 20 + F(2, 0))$

Space Efficiency: $\Theta(n, w)$

Time Efficiency: $\Theta(n, w)$ $\rightarrow$ because there are n, w entries in the table and a constant operation is made for each entry

* Classical DP approach fills a table with solutions to all smaller subproblems, but each of them is solved only once.

Weighted Interval Scheduling

$\rightarrow$ A request takes the form:

Can I reserve the resource starting at time s until time f ?

We assume that the resource can be used by at most 1 person at a time.

A scheduler wants to accept a subset of these requests (rejecting all others) so that the accepted requests do not overlap in time.

$\rightarrow$ We have n requests labeled $1, \dots, n$ with each request i specifying a start time s_i and a finish time f_i

$\rightarrow$ Each interval i now also has a value (or weight) v_i

$\rightarrow$ 2 intervals are compatible if they do not overlap

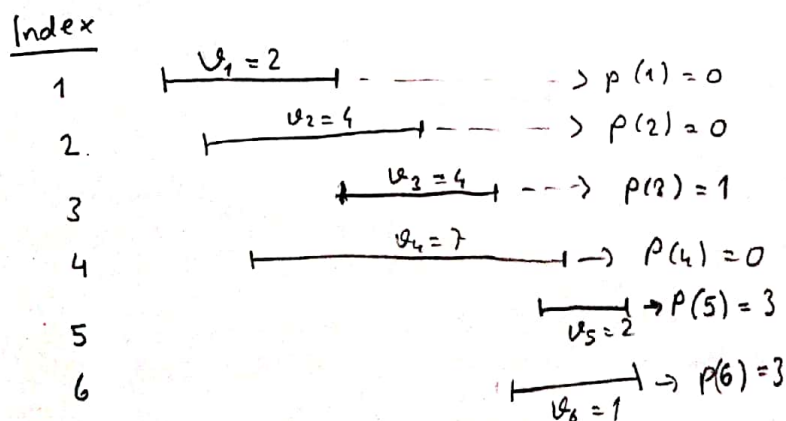
$\rightarrow$ The goal is to select a subset $S \subseteq \{1, \dots, n\}$ of mutually compatible intervals, so as to maximize the sum of the values of the selected intervals

$$\sum_{i \in S} v_i$$

$\rightarrow$ Suppose that the requests are sorted in order of nondecreasing finish time: $f_1 \leq f_2 \leq \dots \leq f_n$

$\rightarrow$ we say that a request i comes from before a request j if $i < j$.
 $p(j)$ = The largest index $i < j$ for an interval i such that intervals i and j are disjoint.

$\bar{p}(i)$: The leftmost interval before i begins



⊗ Consider an optimal solution

O^*

⇒ Either interval n belongs to O^* or it does not

If $n \in O^* \Rightarrow$ No interval indexed strictly between $p(n)$ and n can belong to O^* . (by the definition of $p(n)$)

⇓

Also, then O^* must include an optimum solution to the problem consisting of requests $\{1, \dots, p(n)\}$ because otherwise, we could replace O^* 's choice of requests from $\{1, \dots, p(n)\}$ with a better one, with no danger of overlapping request n .

→ If $n \notin O^*$, then O^* is simply equal to the optimal solution to the problem consisting of requests $\{1, \dots, n-1\}$

→ Let $O^*(j)$ denote the optimal solution to the problem consisting of requests $\{1, \dots, j\}$

Let $OPT(j)$ = the value of this solution
(Define $OPT(0) = 0$)

We are seeking $O^* = OPT(n)$

$$OPT(j) = \max \{u_j + OPT(p(j)), OPT(j-1)\}$$

⇒ Request j belongs to an optimal solution on the set $\{1, 2, \dots, j\}$ if and only if $u_j + OPT(p(j)) \geq OPT(j-1)$

Compute $OPT(j)$

if $j = 0$ then
return 0

else

return $\max(u_j + \text{Compute } OPT(p(j)), \text{Compute } OPT(j-1))$

end if

Greedy Algorithms

→ An algorithm is greedy if it builds up a solution in small steps, choosing a decision at each step myopically to optimize some underlying criterion.

Interval Scheduling

$s(i)$ = Starting time of request i

$f(i)$ = Finishing time of request i

⊗ We say that a subset of the requests is compatible if no 2 of them overlap in time, and our goal is to accept as large a compatible subset as possible.

Basic idea: Use a simple rule to select the first request i_1 .

Reject all requests that are incompatible with i_1

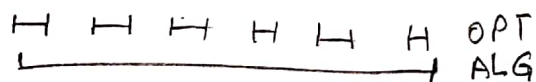
Then select the next request i_2 & again reject the incompatible ones

Continue until we run out of requests

Which simple rule should we use?

Option-1: Select the available request that starts earliest (the one with minimum start time $s(i)$)

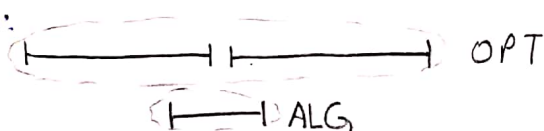
Ex:



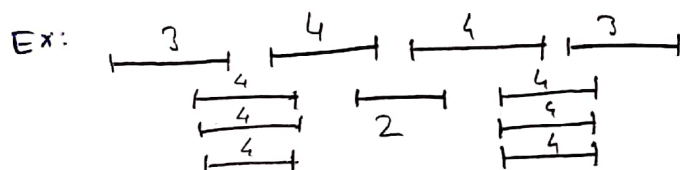
If the earliest request is for a very long interval, ALG fails.

Option-2: Select the request that requires the smallest interval of time (with minimum $f(i) - s(i)$)

Ex:



Option-3: For each request, count the number of other requests that are not compatible and accept the request with fewest number of conflicts.



~~Option-4~~
Optimum option: Accept the request that finishes first (with minimum $f(i)$)